

Field-Level Encryption & Dynamic Masking

Use case: Protect PII in OLTP

Postgres

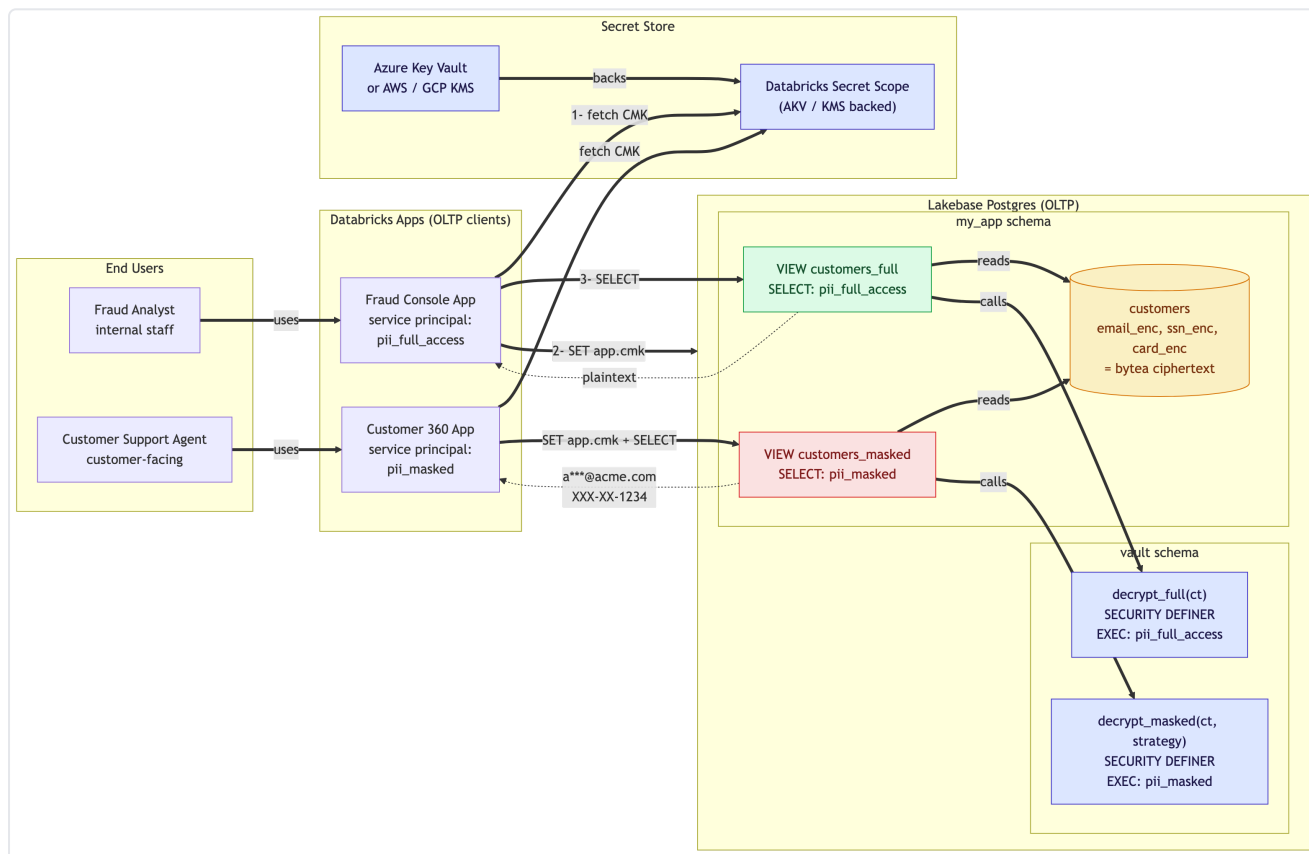
Components: Lakebase · pgcrypto

AKV / KMS · Databricks Apps

Author: Nan Chulpaibul · Field Engineering, Databricks · Date: May 2026 · Version: 1.0

Encryption-at-rest only protects you against a stolen disk. Once a user authenticates, they see plaintext. **Field-level encryption** stores PII as ciphertext in the table and decrypts only inside an authorized function call, so a **customer-managed key (CMK)** from **Azure Key Vault / KMS** controls who sees what. Add **role-based masking** and the same row looks completely different to a fraud analyst vs. a support agent.

Architecture



CMK lives in Azure Key Vault → fetched at session start by the Databricks App → injected into Lakebase as a session GUC → decrypt path is gated per Postgres role

How it works (in order)

1. The **customer-managed key (CMK)** is stored in **Azure Key Vault** (or AWS / GCP KMS) and surfaced to Databricks via an AKV-backed Secret Scope. Every key access is audited.
2. The CMK is injected into the Postgres session via `SELECT set_config('app.cmk', '...');`. The CMK lives only in that connection's session memory; it is never persisted in Postgres.

3. PII columns in the base table are stored as **bytea ciphertext**, encrypted with `pgp_sym_encrypt(plaintext, cmk)`.
4. Two **SECURITY DEFINER** functions decrypt: `my_vault.decrypt_full` returns plaintext, `my_vault.decrypt_masked` returns an already-masked string. Each is granted to a single Postgres role.
5. Two role-specific **views** wire each role to its decrypt path: `customers_full` for `pii_full_access`, `customers_masked` for `pii_masked`. Consumers query the view their role can see.

Try it: paste these SQL blocks into the Lakebase SQL Editor

The blocks below run end-to-end in the Lakebase SQL Editor. Run them top-to-bottom as the project owner. Each block is idempotent — safe to re-run.

STEP 1 — ENABLE `pgcrypto` AND CREATE SCHEMAS

The only Postgres extension required is `pgcrypto`, which is on Lakebase's allowlist by default.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE SCHEMA IF NOT EXISTS my_vault;    -- holds the decrypt functions
CREATE SCHEMA IF NOT EXISTS my_app;      -- holds the encrypted table + views
```

STEP 2 — CREATE THE TWO DECRYPT FUNCTIONS

Both functions read the CMK from `current_setting('app.cmk')`. Marking them `SECURITY DEFINER` lets them be ACL-gated independently from access to the underlying table.

```
CREATE OR REPLACE FUNCTION my_vault.decrypt_full(p_ct bytea)
RETURNS text
LANGUAGE plpgsql SECURITY DEFINER STABLE AS $$
BEGIN
    RETURN public.pgp_sym_decrypt(p_ct, current_setting('app.cmk'));
END; $$;

CREATE OR REPLACE FUNCTION my_vault.decrypt_masked(p_ct bytea, strategy text)
RETURNS text
LANGUAGE plpgsql SECURITY DEFINER STABLE AS $$
DECLARE v text;
BEGIN
    v := public.pgp_sym_decrypt(p_ct, current_setting('app.cmk'));
    RETURN CASE strategy
        WHEN 'email' THEN regexp_replace(v, '^(.*)@.*$', '\1***\2')
        WHEN 'ssn'   THEN 'XXX-XX-' || right(v, 4)
        WHEN 'card'  THEN '*****' || right(v, 4)
        ELSE '***REDACTED***'
    END;
END; $$;
```

STEP 3 — CREATE ROLES AND ACLS

Each role can call exactly one decrypt path. Neither role can read the CMK directly because `SECURITY DEFINER` means the function runs with the owner's privileges.

```

CREATE ROLE pii_full_access;
CREATE ROLE pii_masked;

REVOKE EXECUTE ON FUNCTION my_vault.decrypt_full(bytea) FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION my_vault.decrypt_masked(bytea, text) FROM PUBLIC;

GRANT EXECUTE ON FUNCTION my_vault.decrypt_full(bytea) TO pii_full_access;
GRANT EXECUTE ON FUNCTION my_vault.decrypt_masked(bytea, text) TO pii_masked;

```

STEP 4 — CREATE THE ENCRYPTED BASE TABLE

PII columns are `bytea`. The plaintext never lands in the column.

```

CREATE TABLE my_app.customers (
  id          bigserial PRIMARY KEY,
  full_name   text,
  email_enc   bytea NOT NULL,
  ssn_enc     bytea NOT NULL,
  card_number_enc bytea NOT NULL
);

```

STEP 5 — CREATE THE ROLE-SPECIFIC VIEWS

Two views, one per role. Each calls exactly one decrypt function so Postgres' planner-level permission check is also enforced.

```

CREATE VIEW my_app.customers_full AS
SELECT id, full_name,
       my_vault.decrypt_full(email_enc) AS email,
       my_vault.decrypt_full(ssn_enc) AS ssn,
       my_vault.decrypt_full(card_number_enc) AS card_number
FROM my_app.customers;

CREATE VIEW my_app.customers_masked AS
SELECT id, full_name,
       my_vault.decrypt_masked(email_enc, 'email') AS email,
       my_vault.decrypt_masked(ssn_enc, 'ssn') AS ssn,
       my_vault.decrypt_masked(card_number_enc, 'card') AS card_number
FROM my_app.customers;

GRANT SELECT ON my_app.customers_full TO pii_full_access;
GRANT SELECT ON my_app.customers_masked TO pii_masked;

```

STEP 6 — SET THE CMK, INSERT SAMPLE DATA, QUERY BOTH VIEWS

For the SQL Editor demo, set the CMK manually via `set_config` (session-local — it disappears when the tab closes). Then insert a few rows and read them through each view.

```

-- 6a. Set the CMK for THIS session only (replace with a strong value)
SELECT set_config('app.cmk', 'demo-cmk-replace-with-akv-secret-2026-05', false);

-- 6b. Insert encrypted rows using the session CMK
INSERT INTO my_app.customers(full_name, email_enc, ssn_enc, card_number_enc) VALUES
('Alice Anderson',
pgp_sym_encrypt('alice@acme.com', current_setting('app.cmk')),
pgp_sym_encrypt('123-45-6789', current_setting('app.cmk')),
pgp_sym_encrypt('4111111111111111', current_setting('app.cmk')),
('Bob Brown',
pgp_sym_encrypt('bob@globex.com', current_setting('app.cmk')),
pgp_sym_encrypt('987-65-4321', current_setting('app.cmk')),
pgp_sym_encrypt('55000000000000004', current_setting('app.cmk')));

-- 6c. Read as the privileged role -> plaintext
SET ROLE pii_full_access;
SELECT * FROM my_app.customers_full ORDER BY id;
RESET ROLE;

-- 6d. Read as the masked role -> redacted
SET ROLE pii_masked;
SELECT * FROM my_app.customers_masked ORDER BY id;
RESET ROLE;

```

NOTE — FROM SQL EDITOR DEMO TO PRODUCTION APP

In the SQL Editor demo above, `set_config('app.cmk', '...')` is called manually. **In production, this is done automatically by the consuming Databricks App at connection time:**

1. The app authenticates to Databricks and reads the CMK from **Azure Key Vault** (or AWS / GCP KMS) via a Databricks Secret Scope — every read is audited.
2. The app opens a Lakebase Postgres connection and immediately runs `SELECT set_config('app.cmk', :cmk, false);`.
3. All subsequent queries in that session decrypt seamlessly. The CMK is never persisted in Postgres and never appears in `pg_stat_statements`.

This means the same SQL above works unchanged from a customer 360 microservice, an internal fraud console, or a scheduled job — only the CMK source changes from a hardcoded demo value to an AKV / KMS lookup.

Expected output — same row, different roles

Step 6c · role `pii_full_access` reading `my_app.customers_full`

```

1 | Alice Anderson | alice@acme.com | 123-45-6789 | 4111111111111111
2 | Bob Brown     | bob@globex.com  | 987-65-4321 | 5500000000000004

```

Step 6d · role `pii_masked` reading `my_app.customers_masked`

```

1 | Alice Anderson | a***@acme.com | XXX-XX-6789 | *****1111
2 | Bob Brown     | b***@globex.com | XXX-XX-4321 | *****0004

```

Full repo (SQL · Python demo · diagrams): github.com/nan-databricks/lakebase-field-level-encryption